CSS3

Complete Notes

From Basics to Responsive Design
with Code and Practical Examples

Prepared and Taught by

Kapil

Mentor

Table of Contents

1. Introduction to CSS3 2
2. History of CSS 4
3. Types of CSS 5
4. What is a Selector? 7
5. Simple Selectors 9
6. Combinator Selectors 11
7. Attribute Selectors 12
8. Pseudo-class Selectors 13
9. Pseudo-element Selectors 15
10. What is a Property? 17
11. Color Property 19
12. Background Property 20
13. Font Property 22
14. Text Property 24
15. Vendor Prefix 26
16. CSS Box Model Rule 27
17. Box Sizing Property 29
18. Overflow Property 30
19. Box Shadow Property 31
20. Margin Property 32
21. Padding Property 33
22. Border and Border-radius Property 34
23. Display Property 35
24. Position Property 37
25. Transform Property 38
26. Animation Property 39
27. CSS Units 41
28. Responsive Design using Media Queries 43
Quick Revision Cheat Sheet 46

1. Introduction to CSS3

1.1 What is CSS?

CSS stands for **Cascading Style Sheets**. It is a style sheet language used to describe the presentation (look and layout) of a document written in HTML or XML. HTML gives a web page its **structure and content**, while CSS controls **how that content looks**: colours, fonts, spacing, layout, animations and responsiveness.

- **Cascading** - when several rules target the same element, CSS decides which rule wins using a fixed order (origin, importance, specificity and source order).
- **Style** - it defines visual properties such as `color`, `font-size`, `margin`.
- **Sheet** - the rules are usually saved in a separate `.css` file, like a sheet of instructions.

1.2 What is CSS3?

CSS3 is the latest evolution of CSS. Unlike CSS1 and CSS2, which were single large specifications, CSS3 is split into **modules** (Selectors, Backgrounds and Borders, Color, Fonts, Text, Box Model, Flexbox, Grid, Transforms, Transitions, Animations, Media Queries, etc.). Each module is developed and released independently, so new features reach browsers faster.

1.3 Why do we use CSS?

- **Separation of concerns** - content (HTML) is separate from design (CSS).
- **Reusability** - one CSS file can style hundreds of pages.
- **Easy maintenance** - change one line and the whole website updates.
- **Faster loading** - the browser caches the CSS file once.
- **Responsive design** - same page adapts to mobile, tablet, laptop, TV, print.
- **Accessibility and SEO** - clean HTML with CSS design is easier for screen readers and search engines.

1.4 Major new features in CSS3

Feature	Examples
Rounded corners	`border-radius`
Shadows	`box-shadow`, `text-shadow`
Gradients	`linear-gradient()`, `radial-gradient()`
Multiple backgrounds	`background: url(a.png), url(b.png);`
Transforms and 3D	`transform: rotate(45deg)`, `perspective`
Transitions and animations	`transition`, `@keyframes`, `animation`
Web fonts	`@font-face`
Advanced selectors	`:nth-child()`, `[attr^=val]`, `:not()`
Layout systems	Flexbox, Grid, Multi-column
Media queries	`@media (max-width: 768px)`
Colour models	`rgba()`, `hsl()`, `hsla()`, opacity

1.5 CSS syntax (anatomy of a rule)

```
selector {
    property: value;
    property: value;
}

/* Example */
h1 {
    color: red;          /* property: color , value: red */
    font-size: 32px;     /* property: font-size , value: 32px */
}
```

- **Selector** - tells the browser which HTML element(s) to style (`h1`).
- **Declaration block** - everything inside `{ }`.
- **Declaration** - one `property: value;` pair.
- **Property** - the feature you want to change (`color`).
- **Value** - the setting you want (`red`).
- Each declaration ends with a **semicolon (;)**. Comments are written as `/* comment */`.

■ Practical Example: Your first styled page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First CSS Page</title>
  <style>
    body   { background-color: #f0f4ff; font-family: Arial, sans-serif; }
    h1     { color: #1b2a49; text-align: center; }
    p      { color: #333; font-size: 18px; line-height: 1.6; }
    .box   { background: white; padding: 20px; margin: 30px auto; width: 60%;
             border-radius: 10px; box-shadow: 0 4px 12px rgba(0,0,0,0.2); }
  </style>
</head>
<body>
  <div class="box">
    <h1>Welcome to CSS3</h1>
    <p>This paragraph is styled using CSS.</p>
  </div>
</body>
</html>
```

Result: A light-blue page with a centered white card that has rounded corners and a soft shadow, a dark blue centered heading and readable grey paragraph text.

2. History of CSS

Before CSS, developers used HTML presentation tags such as `<font>`, `<center>` and `<b>` and layout with tables. This mixed content with design and made pages heavy and hard to maintain. CSS was created to solve this problem.

Year	Milestone
1990-1993	Tim Berners-Lee creates the Web (HTML). Early browsers let users set their own styles, but authors had no standard way.
1994	**Hakon Wium Lie** (working at CERN) proposes *Cascading HTML Style Sheets*. Bert Bos later joins the effort.
1995	W3C (World Wide Web Consortium) shows interest and starts a CSS working group.
Dec 1996	**CSS Level 1 (CSS1)** becomes a W3C Recommendation: fonts, colours, margins, text, backgrounds.
1996-97	Internet Explorer 3 is the first mainstream browser with partial CSS support.
May 1998	**CSS Level 2 (CSS2)**: positioning (absolute, fixed), z-index, media types, table layout, bidirectional text.
1999	Work on **CSS3** starts. The specification is broken into independent modules.
2011	**CSS 2.1** becomes the official W3C Recommendation (fixes errors in CSS2).
2011-2012	Major CSS3 modules become stable: Selectors, Media Queries, Color, Namespaces.
2012-2017	Flexbox and Grid Layout mature; Grid gets support in all major browsers in 2017.
Today	CSS is now maintained as a living standard through modules (CSS Snapshot). There is no 'CSS4'; new modules are called Level 4, Level 5, etc. Modern features: variables, container queries, nesting, `:has()`.

Note: After CSS3 the W3C stopped releasing one big version number. Every module has its own level (for example Selectors Level 4, Color Level 5).

3. Types of CSS

There are three ways to add CSS to an HTML document: **Inline**, **Internal (Embedded)** and **External**. (A fourth technique, `@import`, is used inside CSS files to include other CSS files.)

3.1 Inline CSS

CSS is written inside the element's `style` attribute. It affects **only that single element**.

```
<h1 style="color: red; text-align: center;">Inline Styled Heading</h1>
<p style="font-size: 20px; background-color: yellow;">Inline paragraph</p>
```

- **Advantages:** quick testing, highest priority (overrides internal and external), useful for HTML emails and dynamic styles set by JavaScript.
- **Disadvantages:** not reusable, mixes content with design, hard to maintain, cannot style pseudo-classes such as `:hover`, cannot use media queries.

3.2 Internal (Embedded) CSS

CSS is written inside a `<style>` tag in the `<head>` section. It affects **the whole page**.

```
<head>
  <style>
    h1 { color: blue; }
    p  { font-size: 18px; color: #444; }
    .highlight { background: yellow; }
  </style>
</head>
<body>
  <h1>Internal CSS</h1>
  <p class="highlight">Styled from the head section.</p>
</body>
```

- **Advantages:** one file only, can use all selectors, good for single-page designs.
- **Disadvantages:** styles cannot be shared with other pages, increases HTML file size.

3.3 External CSS

CSS is written in a separate `.css` file and linked to the HTML page with the `<link>` tag placed in `<head>`. This is the **best and most professional method**.

```
<!-- index.html -->
<head>
  <link rel="stylesheet" href="style.css">
</head>

/* style.css */
body { font-family: Arial, sans-serif; margin: 0; }
h1   { color: green; }
```

- **Advantages:** one file controls many pages, browser caching makes pages load faster, easy teamwork and maintenance, clean HTML.
- **Disadvantages:** page may show unstyled for a moment if the file is slow to load; extra HTTP request.

3.4 @import rule

```
/* main.css */
@import url("reset.css");
@import "typography.css";
body { color: #333; }
```

Note: `@import` must be at the very top of the stylesheet. It loads files one after another, so `<link>` is faster and preferred.

3.5 Priority (which one wins?)

If the same property is defined in multiple places, the order (from strongest to weakest) is: **Inline CSS > Internal / External CSS (the one written last wins, if specificity is equal) > Browser default style.**

Point	Inline	Internal	External
Location	`style` attribute	`<style>` in head	Separate `.css` file
Scope	One element	One page	Whole website
Reusable	No	Only in that page	Yes
Maintenance	Very hard	Medium	Easy
Recommended	Rarely	Small demos	Always in real projects

■ Practical Example: All three types in one page

```
<link rel="stylesheet" href="style.css">     <!-- style.css: p { color: green; } -->
<style> p { color: blue; } </style>          <!-- internal, comes later -->
<p>Which colour am I?</p>                    <!-- BLUE (internal is later) -->
<p style="color: red;">And me?</p>           <!-- RED (inline wins) -->
```

Result: The first paragraph is blue because the internal rule appears after the external link; the second is red because inline CSS has the highest priority.

4. What is a Selector?

A **selector** is the part of a CSS rule that **selects (targets) the HTML element(s)** you want to style. You can think of it as an address: the selector finds the element, and the declarations inside `{ }` describe how it should look.

4.1 Categories of CSS selectors

Category	Purpose	Example
Simple selectors	Select by tag name, class, id, or all	`p`, `.card`, `#logo`, `*`
Combinator selectors	Select based on relationship between elements	`div p`, `ul > li`, `h1 + p`, `h1 ~ p`
Attribute selectors	Select by attribute or its value	`a[target="_blank"]`, `input[type="text"]`
Pseudo-class selectors	Select an element in a special state	`a:hover`, `li:first-child`
Pseudo-element selectors	Style a part of an element	`p::first-line`, `h1::before`

4.2 Grouping selectors

When several elements share the same style, separate the selectors with a comma to avoid repeating code.

```
h1, h2, h3, .title, #main-heading {
    font-family: Georgia, serif;
    color: #1b2a49;
}
```

4.3 Specificity (how the browser picks a winner)

Every selector gets a weight. The higher weight wins regardless of order. Weight is counted as four levels: **(Inline, IDs, Classes/Attributes/Pseudo-classes, Elements/Pseudo-elements)**.

Selector	Specificity	Explanation
`p`	0-0-0-1	1 element
`.note`	0-0-1-0	1 class
`p.note`	0-0-1-1	1 class + 1 element
`#intro`	0-1-0-0	1 id
`div #intro .note`	0-1-1-1	1 id + 1 class + 1 element
`style="..."`	1-0-0-0	inline style
`!important`	overrides all	avoid using it unless necessary

```
p          { color: black; }   /* weight 0-0-0-1 */
.note      { color: blue;  }   /* weight 0-0-1-0  -> wins over p       */
#intro     { color: red;   }   /* weight 0-1-0-0  -> wins over .note   */
```

4.4 Inheritance

Some properties (like `color`, `font-family`, `line-height`, `text-align`) are **inherited** by child elements; others (like `margin`, `border`, `padding`, `background`) are not.

```
body { color: navy; font-family: Verdana; }   /* all text inside inherits this */
```

5. Simple Selectors

Selector	Syntax	Meaning
Universal	`* { }`	Selects every element on the page
Type / Element	`p { }`	Selects all elements of that tag
Class	`.name { }`	Selects elements with `class="name"`; can be reused many times
ID	`#name { }`	Selects the ONE element with `id="name"`; must be unique per page
Group	`h1, p { }`	Selects multiple selectors at once

5.1 Universal selector (*)

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;   /* very common CSS reset */
}
```

5.2 Type (element) selector

```
h1 { color: crimson; }
p  { font-size: 17px; }
```

5.3 Class selector

A class starts with a **dot (.)**. One element can have several classes separated by space.

```
.btn        { padding: 10px 20px; border: none; border-radius: 6px; }
.btn-blue   { background: royalblue; color: white; }
.btn-red    { background: crimson;  color: white; }

<button class="btn btn-blue">Save</button>
<button class="btn btn-red">Delete</button>
```

5.4 ID selector

An ID starts with a **hash (#)** and must be unique inside a page. IDs are also used for page anchors (`href="#contact"`) and JavaScript.

```
#header { background: #1b2a49; color: white; padding: 20px; }
<header id="header">My Website</header>
```

5.5 Class vs ID

Point	Class	ID
Symbol	. (dot)	# (hash)
Reuse	Many elements	Only one element
Specificity	Lower (0-0-1-0)	Higher (0-1-0-0)
Multiple on one element	Yes	No
Best use	Reusable styling	Unique landmarks, JS hooks, anchors

■ Practical Example: Simple selectors together

```
<style>
  * { font-family: Arial, sans-serif; }
  h2 { color: teal; }
  .card { border: 1px solid #ccc; padding: 15px; margin: 10px; border-radius: 8px; }
  .card.featured { background: #fff3cd; }
  #offer { color: crimson; font-weight: bold; }
</style>

<h2>Products</h2>
<div class="card">Basic Plan</div>
<div class="card featured">Pro Plan <span id="offer">50% OFF</span></div>
```

Result: Both cards get the border and padding; the second card also gets a yellow background because it has two classes; only the word 50% OFF is red and bold.

6. Combinator Selectors

A combinator explains the **relationship between two selectors**. CSS has four combinators.

Combinator	Symbol	Selects	Example
Descendant	space	All matching elements anywhere inside the parent (child, grandchild...)	`div p`
Child	`>`	Only direct children	`ul > li`
Adjacent sibling	`+`	The very next sibling immediately after	`h1 + p`
General sibling	`~`	All following siblings at the same level	`h1 ~ p`

HTML used for the examples below

```
<div class="wrapper">
  <p>Paragraph 1 (direct child)</p>
  <section>
     <p>Paragraph 2 (grandchild)</p>
  </section>
  <h2>Heading</h2>
  <p>Paragraph 3 (right after h2)</p>
  <p>Paragraph 4 (later sibling)</p>
</div>

/* 1. Descendant : P1, P2, P3, P4 (all p inside .wrapper) */
.wrapper p { color: blue; }

/* 2. Child : only P1, P3, P4 (direct children); P2 is NOT selected */
.wrapper > p { font-weight: bold; }

/* 3. Adjacent sibling : only P3 (immediately after h2) */
h2 + p { background: yellow; }

/* 4. General sibling : P3 and P4 (all p after h2) */
h2 ~ p { border-left: 4px solid green; padding-left: 8px; }
```

■ Practical Example: Navigation menu using child combinator

```
<ul class="menu">
  <li>Home</li>
  <li>Services
     <ul><li>Web</li><li>App</li></ul>
  </li>
  <li>Contact</li>
</ul>

.menu > li { display: inline-block; padding: 8px 16px; background: #1b2a49; color: #fff; }
.menu li ul li { background: #2563eb; }   /* sub-menu items only */
```

Result: Only the top-level menu items get the dark-blue style. Items in the nested list are styled separately.

7. Attribute Selectors

Attribute selectors select elements based on the presence or value of an HTML attribute. They are written inside square brackets `[ ]`.

Selector	Meaning	Example
`[attr]`	Has the attribute	`input[required]`
`[attr="value"]`	Attribute equals exact value	`input[type="text"]`
`[attr~="value"]`	Space-separated list contains the word	`[class~="card"]`
`[attr|="value"]`	Value equals or starts with value followed by hyphen	`[lang|="en"]` (en, en-US)
`[attr^="value"]`	Value **begins with**	`a[href^="https"]`
`[attr$="value"]`	Value **ends with**	`a[href$=".pdf"]`
`[attr*="value"]`	Value **contains** the text anywhere	`a[href*="google"]`
`[attr="v" i]`	Case-insensitive match	`[type="TEXT" i]`

■ Practical Example: Form and link styling

```
<input type="text" placeholder="Name">
<input type="email" placeholder="Email" required>
<input type="password" placeholder="Password">
<a href="https://example.com">Secure link</a>
<a href="notes.pdf">Download PDF</a>
<a href="mailto:me@site.com">Email me</a>
<a href="https://site.com" target="_blank">Opens in new tab</a>

input[type="text"], input[type="email"], input[type="password"] {
    border: 2px solid #999; padding: 8px; margin: 4px; border-radius: 4px;
}
input[required]         { border-color: crimson; }
a[href^="https"]        { color: green; }
a[href$=".pdf"]         { color: red; font-weight: bold; }
a[href^="mailto"]       { color: purple; }
a[target="_blank"]::after { content: " \2197"; }
```

Result: Required email box gets a red border; https links are green; the PDF link is bold red; mailto link is purple; the new-tab link gets a small arrow after it.

8. Pseudo-class Selectors

A **pseudo-class** is a keyword added to a selector with a **single colon (:)** to style an element in a **special state** or position (mouse over, focused, first child, checked, etc.).

```
selector:pseudo-class { property: value; }
```

8.1 Link and user-action pseudo-classes

Pseudo-class	When it applies
`:link`	Unvisited link
`:visited`	Link the user has visited
`:hover`	Mouse pointer is over the element
`:active`	Element is being clicked (mouse button pressed)
`:focus`	Element has keyboard/mouse focus (inputs, buttons, links)
`:focus-within`	Element or any descendant has focus
`:focus-visible`	Focus shown only for keyboard navigation

Note: For links, use the order **L-V-H-A**: `:link`, `:visited`, `:hover`, `:active` (remember: LoVe HAte).

8.2 Structural pseudo-classes

Pseudo-class	Meaning
`:first-child` / `:last-child`	First / last child of its parent
`:only-child`	Element that is the only child
`:nth-child(n)`	Nth child. `2` , `odd`, `even`, `3n`, `2n+1`, `-n+3`
`:nth-last-child(n)`	Counting from the end
`:first-of-type` / `:last-of-type`	First / last of that tag among siblings
`:nth-of-type(n)`	Nth element of that tag type
`:empty`	Element with no children or text
`:root`	The `<html>` element (used for CSS variables)
`:not(selector)`	Elements that do NOT match the selector
`:target`	Element whose id matches the URL fragment (#id)

8.3 Form (UI state) pseudo-classes

Pseudo-class	Meaning
`:checked`	Checked checkbox / radio / selected option
`:disabled` / `:enabled`	Disabled or enabled form control
`:required` / `:optional`	Field with or without `required`
`:valid` / `:invalid`	Input value passes / fails validation
`:read-only`	Not editable
`:placeholder-shown`	Input currently showing its placeholder

■ Practical Example: Interactive button, table stripes and form validation

```
/* Links */
a:link    { color: blue; }
a:visited { color: purple; }
a:hover   { color: red; text-decoration: underline; }
a:active  { color: orange; }

/* Button hover effect */
.btn { background: #2563eb; color: #fff; padding: 10px 22px; border: 0; border-radius: 6px;
       transition: 0.3s; cursor: pointer; }
.btn:hover  { background: #1e40af; transform: translateY(-2px); }
.btn:active { transform: translateY(0); }

/* Zebra table */
tr:nth-child(even) { background: #eef3ff; }
tr:nth-child(odd)  { background: #ffffff; }
tr:first-child     { background: #1b2a49; color: white; }

/* List tricks */
li:first-child { font-weight: bold; }
li:last-child  { border-bottom: none; }
li:nth-child(3n) { color: crimson; }        /* 3rd, 6th, 9th ... */
li:not(:last-child) { border-bottom: 1px solid #ddd; }

/* Form */
input:focus   { outline: none; border-color: #2563eb; box-shadow: 0 0 6px #2563eb; }
input:valid   { border-color: green; }
input:invalid { border-color: red; }
input:disabled { background: #eee; cursor: not-allowed; }
input:checked + label { color: green; font-weight: bold; }
```

Result: Buttons lift on hover, table rows alternate colours, invalid inputs get a red border and focused inputs glow blue.

9. Pseudo-element Selectors

A **pseudo-element** styles a **specific part** of an element or inserts virtual content. It uses a **double colon (::)** in CSS3 (single colon is accepted for older ones for backward compatibility).

Pseudo-element	What it styles
`::before`	Inserts content before the element's content (needs `content`)
`::after`	Inserts content after the element's content (needs `content`)
`::first-letter`	The first letter of a block element (drop caps)
`::first-line`	The first line of text of a block element
`::selection`	Text highlighted by the user
`::placeholder`	Placeholder text of an input
`::marker`	Bullet or number of a list item
`::backdrop`	Background behind a fullscreen element or dialog

Note: `::before` and `::after` do nothing unless the `content` property is set (even `content: "";` for empty decoration).

```
/* Drop cap */
p::first-letter { font-size: 48px; font-weight: bold; color: crimson;
                  float: left; padding-right: 6px; }

/* First line */
p::first-line { font-variant: small-caps; color: #1b2a49; }

/* Selected text colour */
::selection { background: #2563eb; color: white; }

/* Placeholder */
input::placeholder { color: #999; font-style: italic; }

/* List bullets */
li::marker { color: crimson; font-size: 1.2em; }
```

■ Practical Example: Decorations with ::before and ::after

```
<h2 class="title">Our Services</h2>
<p class="required-field">Email</p>
<a class="ext" href="https://example.com">Visit site</a>
<blockquote>Practice makes perfect.</blockquote>

.title::before { content: "\2605 "; color: gold; }              /* star before */
.title::after  { content: ""; display: block; width: 60px; height: 4px;
                 background: #2563eb; margin-top: 6px; }        /* underline bar */
.required-field::after { content: " *"; color: red; }
.ext::after { content: " \2192"; }
blockquote::before { content: "\201C"; font-size: 40px; color: #999; }
blockquote::after  { content: "\201D"; font-size: 40px; color: #999; }
```

Result: The heading gets a gold star and a blue bar below it, required labels show a red asterisk, and the quote is wrapped with decorative quotation marks - all without adding extra HTML.

Pseudo-class vs Pseudo-element

Point	Pseudo-class	Pseudo-element
Symbol	:	::
Purpose	Selects an element in a state	Selects a part of an element / creates content

Point	Pseudo-class	Pseudo-element
Example	`a:hover`	`p::first-line`

10. What is a Property?

A **CSS property** is the specific style feature of an element that you want to change (for example `color`, `width`, `margin`). Every property is paired with a **value** using a colon, and written as `property: value;` - this pair is called a **declaration**.

```
p {
    color: blue;          /* property = color,       value = blue   */
    font-size: 18px;      /* property = font-size,   value = 18px   */
    margin: 10px 20px;    /* property = margin,      value = 10px 20px */
}
```

10.1 Common groups of properties

Group	Examples
Text and font	`color`, `font-family`, `font-size`, `text-align`, `line-height`
Box model	`width`, `height`, `margin`, `padding`, `border`, `box-sizing`
Background	`background-color`, `background-image`, `background-size`
Layout	`display`, `position`, `float`, `flex`, `grid`, `z-index`
Effects	`box-shadow`, `opacity`, `transform`, `transition`, `filter`
Animation	`animation`, `@keyframes`
List / table	`list-style`, `border-collapse`, `caption-side`

10.2 Shorthand properties

A shorthand lets you set many related properties in a single line.

```
margin: 10px 20px 10px 20px;                  /* top right bottom left */
border: 2px solid red;                        /* width style color */
background: #fff url(bg.png) no-repeat center; /* colour image repeat position */
font: italic bold 16px/1.5 Arial, sans-serif;  /* style weight size/line-height family */
```

10.3 Special (global) values

Value	Meaning
`inherit`	Take the value from the parent element
`initial`	Reset to the CSS default value
`unset`	Inherit if the property is inheritable, otherwise initial
`revert`	Roll back to the browser's default stylesheet
`!important`	Gives the declaration top priority (use sparingly)

10.4 Custom properties (CSS variables)

```
:root {
    --main-color: #2563eb;
    --radius: 8px;
}
.btn {
    background: var(--main-color);
    border-radius: var(--radius);
    color: var(--text, white);      /* fallback value if --text is not defined */
}
```

Change `--main-color` in one place and every element using it updates - perfect for themes and dark mode.

11. Color Property

The `color` property sets the **text (foreground) colour**. Colours can also be used in `background-color`, `border-color`, `box-shadow`, `text-shadow`, `outline-color` and more. CSS supports several ways to define a colour.

11.1 Ways to write colours

Method	Syntax	Example
Colour name	140 predefined names	`red`, `tomato`, `navy`, `gold`
HEX	`#RRGGBB` (00 to FF)	`#ff0000` = red
Short HEX	`#RGB` (each digit doubled)	`#f00` = `#ff0000`
HEX with alpha	`#RRGGBBAA`	`#ff000080` (50% transparent red)
RGB	`rgb(red, green, blue)` 0-255	`rgb(255, 0, 0)`
RGBA	`rgba(r, g, b, alpha)` alpha 0-1	`rgba(255, 0, 0, 0.5)`
HSL	`hsl(hue, saturation, lightness)`	`hsl(0, 100%, 50%)` = red
HSLA	`hsla(h, s, l, alpha)`	`hsla(120, 60%, 40%, 0.7)`
Special keywords	`transparent`, `currentColor`	`border-color: currentColor;`

11.2 Understanding each model

- **HEX**: three pairs of hexadecimal digits for Red, Green and Blue. `#000000` is black, `#ffffff` is white.
- **RGB**: the amount of red, green, blue from 0 to 255. `rgb(0,0,255)` is pure blue.
- **HSL**: Hue is a colour-wheel angle (0 = red, 120 = green, 240 = blue), Saturation is the colour intensity (0% grey to 100% vivid), Lightness (0% black, 50% normal, 100% white). Easy to create shades: just change lightness.
- **Alpha**: transparency. `0` is fully transparent, `1` is fully opaque.
- `opacity` (0 to 1) makes the **entire element** including its children transparent, but `rgba()` makes only that colour transparent.

■ Practical Example: Colour demonstration

```
<h1 class="c1">Named colour</h1>
<h1 class="c2">HEX colour</h1>
<h1 class="c3">RGB colour</h1>
<h1 class="c4">HSL colour</h1>
<div class="overlay"><p>Semi-transparent box</p></div>

.c1 { color: tomato; }
.c2 { color: #2563eb; }
.c3 { color: rgb(15, 118, 110); }
.c4 { color: hsl(280, 70%, 45%); }

body { background: #f0f4ff; }
.overlay { background-color: rgba(27, 42, 73, 0.75); color: white; padding: 20px; }

/* Shades of one colour with HSL: only lightness changes */
.shade1 { background: hsl(220, 80%, 25%); }
.shade2 { background: hsl(220, 80%, 45%); }
.shade3 { background: hsl(220, 80%, 75%); }
```

Result: Four headings in four different colours and a navy box that lets the page background show through slightly.

12. Background Property

Background properties control the area behind an element's content and padding (and border, depending on `background-clip`).

Property	Purpose	Common values
`background-color`	Solid colour	`red`, `#fff`, `rgba()`, `transparent`
`background-image`	Image or gradient	`url("img.jpg")`, `linear-gradient(...)`
`background-repeat`	Tiling	`repeat`, `no-repeat`, `repeat-x`, `repeat-y`, `space`, `round`
`background-position`	Placement	`center`, `top right`, `20px 40px`, `50% 50%`
`background-size`	Image size	`auto`, `cover`, `contain`, `100px 200px`, `50%`
`background-attachment`	Scroll behaviour	`scroll`, `fixed` (parallax), `local`
`background-origin`	Where positioning starts	`padding-box`, `border-box`, `content-box`
`background-clip`	Where background is painted	`border-box`, `padding-box`, `content-box`, `text`
`background-blend-mode`	Blend layers	`multiply`, `overlay`, `screen`
`background`	Shorthand for all	see below

Note: `cover` scales the image to fill the whole box (may crop). `contain` scales the image so it fits fully inside (may leave empty space).

12.1 Shorthand order

```
background: color image repeat attachment position / size;

background: #f0f4ff url("hero.jpg") no-repeat fixed center / cover;
```

12.2 Gradients

```
background: linear-gradient(to right, #2563eb, #9333ea);          /* left to right */
background: linear-gradient(135deg, #f97316 0%, #ec4899 50%, #8b5cf6 100%);
background: radial-gradient(circle at center, #fde68a, #f59e0b);
background: conic-gradient(red, yellow, green, blue, red);        /* pie / colour wheel */
background: repeating-linear-gradient(45deg, #eee 0 10px, #ddd 10px 20px);  /* stripes */
```

12.3 Multiple backgrounds

```
.hero {
    background:
        linear-gradient(rgba(0,0,0,0.5), rgba(0,0,0,0.5)),   /* dark overlay on top */
        url("mountain.jpg") center / cover no-repeat;         /* picture below */
}
```

■ Practical Example: Hero banner with overlay and gradient text button

```
<section class="hero">
  <h1>Learn CSS3</h1>
  <a href="#" class="cta">Get Started</a>
</section>

.hero {
    height: 60vh;
    background: linear-gradient(rgba(0,0,0,.55), rgba(0,0,0,.55)),
                url("banner.jpg") center / cover no-repeat fixed;
    color: white; text-align: center; padding-top: 18vh;
```

```
}
.cta {
    display: inline-block; padding: 12px 30px; border-radius: 30px; color: white;
    text-decoration: none;
    background: linear-gradient(90deg, #2563eb, #9333ea);
}
.gradient-text {           /* gradient coloured text */
    background: linear-gradient(90deg, #f97316, #ec4899);
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
}
```

Result: A full-width banner image darkened by an overlay, fixed while scrolling (parallax), with a purple-blue pill button.

13. Font Property

Property	Purpose	Values
`font-family`	Typeface list (fallbacks)	`"Poppins", Arial, sans-serif`
`font-size`	Size of text	`16px`, `1.2rem`, `120%`, `large`
`font-weight`	Boldness	`normal(400)`, `bold(700)`, `100-900`
`font-style`	Slant	`normal`, `italic`, `oblique`
`font-variant`	Small caps	`normal`, `small-caps`
`line-height`	Vertical space of each line	`1.5`, `24px`, `150%`
`font-stretch`	Condensed / expanded	`condensed`, `expanded`
`font`	Shorthand	style variant weight size/line-height family

13.1 Generic font families

Generic name	Description	Example fonts
`serif`	Small strokes at letter ends	Times New Roman, Georgia
`sans-serif`	Clean, no strokes	Arial, Helvetica, Verdana
`monospace`	Equal-width letters (code)	Courier New, Consolas
`cursive`	Handwriting style	Comic Sans MS, Brush Script
`fantasy`	Decorative	Impact, Papyrus

> **Note:** Always give a fallback list. If the first font is missing, the browser tries the next. Font names with spaces must be in quotes.

13.2 Web fonts

```
<!-- Method 1 : Google Fonts (put in <head>) -->
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600&display=swap"
      rel="stylesheet">
<style> body { font-family: "Poppins", sans-serif; } </style>

/* Method 2 : @font-face with your own file */
@font-face {
    font-family: "MyFont";
    src: url("fonts/MyFont.woff2") format("woff2"),
         url("fonts/MyFont.woff") format("woff");
    font-weight: normal;
    font-display: swap;
}
body { font-family: "MyFont", Arial, sans-serif; }
```

13.3 Font shorthand

```
/* font: [style] [variant] [weight] size[/line-height] family */
p { font: italic small-caps bold 18px/1.6 "Georgia", serif; }
```

> **Note:** In the shorthand, **size and family are mandatory**, and their order must be preserved.

■ Practical Example: Typography for a blog article

```
body { font-family: "Segoe UI", Roboto, Arial, sans-serif; font-size: 16px; line-height: 1.7; }
h1   { font-size: 2.4rem; font-weight: 800; font-family: Georgia, serif; }
h2   { font-size: 1.6rem; font-weight: 600; }
.lead { font-size: 1.25rem; font-style: italic; color: #555; }
code  { font-family: "Courier New", monospace; background: #eee; padding: 2px 5px; }
.caps { font-variant: small-caps; letter-spacing: 1px; }
```

Result: A clean readable article: serif heading, italic lead paragraph, monospace code snippets.

14. Text Property

Property	Purpose	Values / Example
`color`	Text colour	`#333`
`text-align`	Horizontal alignment	`left`, `right`, `center`, `justify`
`text-align-last`	Alignment of last line	`center`
`text-decoration`	Line on text	`none`, `underline`, `overline`, `line-through`
`text-decoration` (full)	line style colour thickness	`underline wavy red 2px`
`text-transform`	Letter case	`uppercase`, `lowercase`, `capitalize`
`text-indent`	First line indent	`30px`
`letter-spacing`	Space between letters	`2px`
`word-spacing`	Space between words	`10px`
`line-height`	Line spacing	`1.6`
`text-shadow`	Shadow behind text	`2px 2px 5px gray`
`white-space`	Spaces and wrapping	`normal`, `nowrap`, `pre`, `pre-wrap`
`text-overflow`	Overflowing text	`clip`, `ellipsis`
`word-break` / `overflow-wrap`	Breaking long words	`break-all`, `break-word`
`vertical-align`	Inline vertical position	`middle`, `top`, `sub`, `super`
`direction`	Text direction	`ltr`, `rtl`

14.1 Truncating text with an ellipsis

```
.title {
    width: 200px;
    white-space: nowrap;       /* keep on one line */
    overflow: hidden;          /* hide extra text  */
    text-overflow: ellipsis;   /* show ...         */
}

/* Multi-line clamp (3 lines) */
.desc {
    display: -webkit-box;
    -webkit-line-clamp: 3;
    -webkit-box-orient: vertical;
    overflow: hidden;
}
```

■ Practical Example: Styled text block

```
h1 {
    text-align: center;
    text-transform: uppercase;
    letter-spacing: 4px;
    text-shadow: 2px 2px 6px rgba(0,0,0,0.35);
    text-decoration: underline wavy #2563eb;
}
p {
    text-align: justify;
    text-indent: 30px;
    line-height: 1.8;
    word-spacing: 3px;
}
a { text-decoration: none; }                /* removes link underline    */
```

```
a:hover { text-decoration: underline; }
.old-price { text-decoration: line-through; color: gray; }
.glow { color: #fff; text-shadow: 0 0 5px #0ff, 0 0 15px #0ff, 0 0 30px #00f; }
```

Result: Centered uppercase spaced-out title with wavy blue underline and shadow; justified paragraphs with a first-line indent; a neon glowing text effect.

15. Vendor Prefix

A **vendor prefix** is a special label added at the start of a CSS property or value so that a specific browser engine can support an **experimental or not-yet-standard feature**. Browser makers used them to test new CSS features before the W3C finalised the standard.

Prefix	Browser engine	Browsers
`-webkit-`	WebKit / Blink	Chrome, Safari, Edge (new), Opera, iOS browsers
`-moz-`	Gecko	Firefox
`-ms-`	Trident	Internet Explorer, old Edge
`-o-`	Presto	Old Opera

```
.box {
    -webkit-transition: all 0.5s;      /* Chrome, Safari */
    -moz-transition: all 0.5s;         /* Firefox */
    -o-transition: all 0.5s;           /* old Opera */
    transition: all 0.5s;              /* standard - ALWAYS LAST */

    -webkit-transform: rotate(45deg);
    -ms-transform: rotate(45deg);
    transform: rotate(45deg);

    -webkit-box-shadow: 0 0 10px black;
    box-shadow: 0 0 10px black;
}

@-webkit-keyframes spin {
    from { -webkit-transform: rotate(0); }
    to   { -webkit-transform: rotate(360deg); }
}
@keyframes spin { from { transform: rotate(0); } to { transform: rotate(360deg); } }
```

- Write the **standard (unprefixed) property last**, so modern browsers use it.
- Prefixes are still useful today for a few properties: `-webkit-line-clamp`, `-webkit-background-clip: text`, `-webkit-appearance`, `-webkit-text-stroke`, `-webkit-backdrop-filter` (older Safari), `::-webkit-scrollbar`.
- Do not write prefixes by hand in big projects: use **Autoprefixer** (PostCSS tool) with a `browserslist` config - it adds the needed prefixes automatically.
- Check support for any feature at **caniuse.com**.

■ Practical Example: Custom scrollbar and gradient text (WebKit-only features)

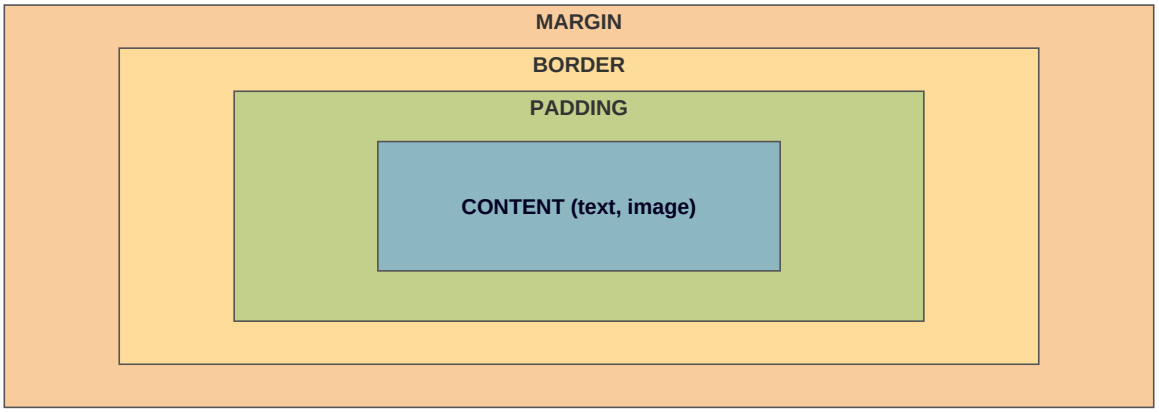
```
.scroll-box { height: 150px; overflow-y: scroll; }
.scroll-box::-webkit-scrollbar        { width: 10px; }
.scroll-box::-webkit-scrollbar-track  { background: #eee; }
.scroll-box::-webkit-scrollbar-thumb  { background: #2563eb; border-radius: 10px; }

.stroke { -webkit-text-stroke: 2px navy; color: transparent; font-size: 60px; }
```

Result: A scroll box with a slim blue rounded scrollbar (Chrome/Safari/Edge) and outlined hollow text.

16. CSS Box Model Rule

In CSS **every HTML element is a rectangular box**. The box model describes how the size and spacing of this box is calculated. From inside to outside it has four layers:



Layer	Description	Property
Content	Actual text/image. Size set by width and height	`width`, `height`
Padding	Transparent space between content and border (takes the element's background)	`padding`
Border	Line around padding and content	`border`
Margin	Transparent space outside the border, separates elements. Never has a background	`margin`

16.1 Calculating the total size (content-box model)

```
Total width  = width + padding-left + padding-right + border-left + border-right
               + margin-left + margin-right
Total height = height + padding-top + padding-bottom + border-top + border-bottom
               + margin-top + margin-bottom

.box {
    width: 300px;
    height: 100px;
    padding: 20px;
    border: 5px solid black;
    margin: 10px;
}
/* Visible box width  = 300 + 20+20 + 5+5   = 350px
   Space it occupies  = 350 + 10+10          = 370px
   Visible box height = 100 + 20+20 + 5+5   = 150px
   Space it occupies  = 150 + 10+10          = 170px          */
```

Note: By default (`box-sizing: content-box`) padding and border are **added** to width/height. This surprises beginners; see next topic for `border-box`.

■ Practical Example: Card with all four layers

```
<div class="card">Box Model</div>

.card {
    width: 250px;
    padding: 25px;
    border: 4px dashed #2563eb;
    margin: 40px auto;
    background: #dbeafe;      /* background fills content + padding (and under border) */
    text-align: center;
}
```

Result: A light-blue card, with a dashed border, centered horizontally, 40px away from other elements. Open browser DevTools (F12) > Elements > Computed tab to see the coloured box-model diagram of any element.

17. Box Sizing Property

`box-sizing` decides **what** `width` **and** `height` **include**.

Value	width/height include	Actual box width for width:300px; padding:20px; border:5px
`content-box` (default)	Content only	300 + 40 + 10 = **350px**
`border-box`	Content + padding + border	**300px** (content shrinks to 250px)
`padding-box`	Content + padding (old Firefox only, removed)	-

```
.a { box-sizing: content-box; width: 300px; padding: 20px; border: 5px solid; }  /* 350px */
.b { box-sizing: border-box;  width: 300px; padding: 20px; border: 5px solid; }  /* 300px */
```

17.1 Best practice: global reset

```
*, *::before, *::after {
    box-sizing: border-box;
}
```

With `border-box`, two columns of `width: 50%` with padding and border still fit exactly side by side.

■ Practical Example: Two columns that do not break the layout

```
<div class="col">Left</div><div class="col">Right</div>

.col { display: inline-block; width: 50%; padding: 20px; border: 2px solid #333;
       vertical-align: top; }
/* content-box  : each column becomes 50% + 44px  -> wraps to a new line (bad)
   border-box   : each column stays exactly 50%   -> fits side by side (good) */
*, *::before, *::after { box-sizing: border-box; }
```

Note: Inline elements (span, a) ignore width/height and vertical margin. Use `display: inline-block` or `block`.

18. Overflow Property

`overflow` controls what happens when content is **larger than its container** (fixed width/height).

Value	Behaviour
`visible` (default)	Content spills outside the box
`hidden`	Extra content is clipped (cut off), no scrollbar
`scroll`	Scrollbars are always shown
`auto`	Scrollbars appear only when needed (most used)
`clip`	Like hidden but no scrolling of any kind, even by script

- `overflow-x` and `overflow-y` control horizontal and vertical overflow separately.
- `overflow: hidden` on a parent also **clears floats** and contains child margins; it is used for rounded-corner image cards.
- Related properties: `text-overflow`, `overscroll-behavior`, `scroll-behavior: smooth`, `scrollbar-width`.

■ Practical Example: Scrollable box, hidden overflow, and image card

```
<div class="v">visible</div> <div class="h">hidden</div>
<div class="s">scroll</div> <div class="a">auto</div>

.v, .h, .s, .a { width: 180px; height: 70px; border: 2px solid #2563eb; margin: 10px;
                 display: inline-block; vertical-align: top; }
/* Put long text (about 40 words) inside each of them */
.v { overflow: visible; }   /* text leaks out below the box */
.h { overflow: hidden;  }   /* text is cut off */
.s { overflow: scroll;  }   /* both scrollbars, always visible */
.a { overflow: auto;    }   /* vertical scrollbar appears only as needed */

/* Chat / comment window */
.chat { height: 300px; overflow-y: auto; overflow-x: hidden; scroll-behavior: smooth; }

/* Rounded image card: image corners are clipped by parent */
.card { width: 300px; border-radius: 16px; overflow: hidden; }
.card img { width: 100%; display: block; }

/* Horizontal scroll strip / responsive table wrapper */
.strip { display: flex; gap: 10px; overflow-x: auto; }
.table-wrap { overflow-x: auto; }
```

Result: The four boxes show four different behaviours for the same overflowing text; the chat area gets a vertical scrollbar; the card image follows the rounded corners.

19. Box Shadow Property

`box-shadow` adds one or more shadows around an element's box (follows `border-radius`).

```
box-shadow: [inset] offset-x offset-y blur-radius spread-radius color;
```

Part	Meaning
`offset-x`	Horizontal distance. Positive = right, negative = left (**required**)
`offset-y`	Vertical distance. Positive = down, negative = up (**required**)
`blur-radius`	Softness. `0` = sharp edge, larger = softer (optional, default 0, no negatives)
`spread-radius`	Expands (+) or shrinks (-) the shadow size (optional)
`color`	Shadow colour; use `rgba()` for soft shadows (optional)
`inset`	Draws the shadow inside the box instead of outside (optional)

```
.s1 { box-shadow: 5px 5px 10px gray; }                    /* basic */
.s2 { box-shadow: 0 4px 12px rgba(0,0,0,0.25); }          /* soft card shadow */
.s3 { box-shadow: 0 0 0 4px #2563eb; }                    /* ring / outline effect */
.s4 { box-shadow: inset 0 0 10px rgba(0,0,0,0.5); }       /* inner shadow */
.s5 { box-shadow: 0 0 10px red, 0 0 20px orange, 0 0 30px yellow; }  /* multiple */
.s6 { box-shadow: 10px 10px 0 #1b2a49; }                  /* solid retro shadow */
```

■ Practical Example: Card that lifts on hover

```
<div class="card">Hover me</div>

.card {
    width: 260px; padding: 30px; margin: 40px; border-radius: 12px; background: white;
    box-shadow: 0 2px 6px rgba(0,0,0,0.15);
    transition: box-shadow 0.3s, transform 0.3s;
}
.card:hover {
    transform: translateY(-8px);
    box-shadow: 0 16px 32px rgba(0,0,0,0.25);
}
/* Input focus glow */
input:focus { outline: none; box-shadow: 0 0 0 3px rgba(37,99,235,.4); }
```

Result: The card floats upward and the shadow becomes larger and softer when you move your mouse over it.

Note: Do not confuse with `text-shadow` (for text: `x y blur color`, no spread/inset) and `filter: drop-shadow()` (follows transparent shape of PNG/SVG).

20. Margin Property

Margin is the transparent space **outside** the border. It pushes other elements away. It never has a background colour.

Property	Description
`margin-top`, `margin-right`, `margin-bottom`, `margin-left`	Individual sides
`margin`	Shorthand for all four sides
`margin-inline`, `margin-block`	Logical properties (left/right, top/bottom in writing direction)

20.1 Shorthand syntax

```
margin: 10px;                  /* all four sides                      */
margin: 10px 20px;             /* top+bottom = 10 , left+right = 20  */
margin: 10px 20px 30px;        /* top = 10 , left+right = 20 , bottom = 30 */
margin: 10px 20px 30px 40px;   /* top right bottom left  (clockwise: TRouBLe) */
margin: 0 auto;                /* vertical 0, horizontally centered  */
```

20.2 Values allowed

- Length: `px`, `em`, `rem`, `vh`...
- Percentage: relative to the **width of the parent** (even for top/bottom).
- `auto`: browser calculates. With a fixed `width`, `margin: 0 auto` **centers a block element**.
- **Negative values** are allowed (`margin-top: -20px`) and pull an element closer or make it overlap.

20.3 Margin collapsing

Vertical margins of adjacent block elements **collapse**: the larger margin wins, they are not added. Example: paragraph A has `margin-bottom: 30px` and paragraph B has `margin-top: 20px`; the gap is **30px**, not 50px. Margins do not collapse for flex/grid items, floats, absolute elements, or when there is padding/border between parent and child.

■ Practical Example: Centered layout and spacing

```
<div class="container">
  <h1>Title</h1>
  <p class="a">Paragraph A</p>
  <p class="b">Paragraph B</p>
</div>

.container { width: 80%; max-width: 900px; margin: 40px auto; background: #eef3ff; }
h1  { margin: 0 0 20px 0; }
.a  { margin-bottom: 30px; }
.b  { margin-top: 20px; }              /* gap between A and B is 30px (collapsed) */
.pull-up { margin-top: -15px; }        /* negative margin */
.push-right { margin-left: auto; }     /* in flex container pushes the item to the right */
```

Result: The container stays horizontally centered at any screen width, and the gap between the two paragraphs is 30px because of margin collapsing.

21. Padding Property

Padding is the space **inside** the border, between the border and the content. It takes the background colour of the element. It increases the clickable area of buttons and links.

```
padding: 20px;                /* all sides                        */
padding: 10px 30px;           /* top/bottom 10 , left/right 30   */
padding: 5px 10px 15px;       /* top 5 , left/right 10 , bottom 15 */
padding: 5px 10px 15px 20px;  /* top right bottom left           */
padding-top: 10px;  padding-right: 20px;  padding-bottom: 10px;  padding-left: 20px;
```

Point	Margin	Padding
Position	Outside the border	Inside the border
Background	Never coloured	Gets the element's background
Negative values	Allowed	**Not** allowed
`auto`	Allowed	Not allowed
Collapsing	Vertical margins can collapse	Never collapses
Percentage	Of parent's width	Of parent's width
Use for	Space between elements	Space between content and border

Note: Padding is added to the width in `content-box`; use `box-sizing: border-box` to avoid size problems.

■ Practical Example: Buttons and a padded alert box

```
<button class="small">Small</button>
<button class="large">Large</button>
<div class="alert">Your profile was updated.</div>

button { border: 0; background: #2563eb; color: #fff; border-radius: 6px; cursor: pointer; }
.small { padding: 6px 12px;  font-size: 13px; }
.large { padding: 16px 40px; font-size: 18px; }
.alert { padding: 15px 20px; background: #dcfce7; border-left: 5px solid #16a34a;
         color: #166534; margin: 20px 0; }

/* Aspect-ratio trick with percentage padding (16:9 box) */
.video { position: relative; padding-top: 56.25%; }
```

Result: Two buttons of different sizes without touching width/height, and a green alert box with comfortable inner spacing.

22. Border and Border-radius Property

22.1 Border

A border is a line drawn around the padding and content. It needs at least a **style** to appear.

Property	Values
`border-width`	`thin`, `medium`, `thick`, or `1px`, `3px`...
`border-style`	`none`, `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `hidden`
`border-color`	any colour value
`border`	shorthand: `width style color`
`border-top`, `border-right`, `border-bottom`, `border-left`	one side (each has width/style/color versions)
`border-collapse`	for tables: `collapse` merges cell borders
`outline`	like border but is **outside** the box and takes no space (used for focus)

```
border: 2px solid #333;
border-top: 4px dashed red;
border-style: solid dotted double dashed;   /* top right bottom left */
border-width: 1px 2px 3px 4px;
border-color: red green blue orange;
border-bottom: 3px solid #2563eb;           /* underline effect */
```

22.2 Border-radius

`border-radius` rounds the corners of the element (and its background and image).

```
border-radius: 10px;                /* all corners                                  */
border-radius: 10px 20px;           /* top-left+bottom-right = 10 , top-right+bottom-left = 20 */
border-radius: 10px 20px 30px 40px; /* top-left, top-right, bottom-right, bottom-left */
border-radius: 50%;                 /* circle (on a square) / ellipse (on rectangle) */
border-radius: 999px;               /* pill / capsule shape                         */
border-radius: 50px / 20px;         /* elliptical: horizontal / vertical radius     */
border-top-left-radius: 20px;       /* single corner                                */
```

■ Practical Example: Circle avatar, pill button, speech bubble, leaf shape

```
<img class="avatar" src="me.jpg" alt="me">
<button class="pill">Subscribe</button>
<div class="bubble">Hello!</div>
<div class="leaf"></div>

.avatar { width: 120px; height: 120px; border-radius: 50%; border: 4px solid #2563eb;
          object-fit: cover; }
.pill   { padding: 10px 28px; border: 2px solid #2563eb; border-radius: 999px;
          background: transparent; color: #2563eb; }
.bubble { display: inline-block; padding: 15px 25px; background: #2563eb; color: white;
          border-radius: 20px 20px 20px 0; }
.leaf   { width: 120px; height: 120px; background: green; border-radius: 0 100% 0 100%; }
table   { border-collapse: collapse; }
td, th  { border: 1px solid #999; padding: 8px; }
```

Result: A round profile picture with a blue ring, a capsule-shaped outline button, a chat bubble with a sharp bottom-left corner and a leaf-shaped green box.

23. Display Property

`display` decides **how an element behaves in the layout**: whether it starts on a new line, whether width/height apply, and how its children are arranged.

Value	Behaviour
`block`	Starts on a new line, takes full available width; width, height, margin, padding all work. (div, p, h1-h6, ul, section)
`inline`	Stays in the line of text; width/height and vertical margins are ignored. (span, a, strong, em)
`inline-block`	Flows inline like text but accepts width, height, margin and padding.
`none`	Element is removed completely from the page (takes no space).
`flex`	Block-level flex container: children (flex items) are arranged in a row/column.
`inline-flex`	Inline version of flex.
`grid`	Two-dimensional layout with rows and columns.
`table`, `table-row`, `table-cell`	Makes elements behave like table parts.
`list-item`	Behaves like `<li>` (shows a bullet).
`contents`	Element's box disappears but its children remain.

23.1 display: none vs visibility: hidden vs opacity: 0

Property	Visible?	Takes space?	Clickable?
`display: none`	No	No	No
`visibility: hidden`	No	**Yes**	No
`opacity: 0`	No	**Yes**	**Yes**

23.2 Flexbox quick guide

```
.flex {
    display: flex;
    flex-direction: row;              /* row | column | row-reverse | column-reverse */
    /* main axis: flex-start|center|flex-end|space-around|space-evenly */
    justify-content: space-between;
    /* cross axis: stretch|flex-start|center|flex-end|baseline */
    align-items: center;
    flex-wrap: wrap;                  /* allow items to go to next line */
    gap: 20px;                        /* space between items */
}
.item { flex: 1; }                    /* grow equally */
```

23.3 Grid quick guide

```
.grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);   /* 3 equal columns */
    gap: 20px;
}
.wide { grid-column: span 2; }
```

■ Practical Example: Navbar, centered box and card grid

```
<nav class="nav"><span class="logo">MySite</span>
  <ul><li>Home</li><li>About</li><li>Contact</li></ul></nav>
<div class="grid"><div class="c">1</div><div class="c">2</div><div class="c">3</div></div>

.nav  { display: flex; justify-content: space-between; align-items: center;
        background: #1b2a49; color: white; padding: 14px 24px; }
.nav ul { display: flex; gap: 24px; list-style: none; margin: 0; padding: 0; }
.center-anything { display: flex; justify-content: center; align-items: center; height: 100vh; }
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; padding: 20px; }
.c    { background: #dbeafe; padding: 40px; text-align: center; border-radius: 8px; }
.hidden-el { display: none; }
```

Result: A horizontal navbar with the logo on the left and links on the right, followed by three equal-width cards.

24. Position Property

`position` defines how an element is placed in the page. Together with `top`, `right`, `bottom`, `left` (offset properties) and `z-index` it lets you move elements from their normal place.

Value	Behaviour	Offsets (top/left..)
`static` (default)	Normal document flow	Ignored
`relative`	Stays in flow; offsets move it **relative to its own original position**; original space is kept. Also becomes reference for absolute children.	Work
`absolute`	Removed from flow; positioned relative to the **nearest positioned ancestor** (non-static), or the page if none.	Work
`fixed`	Removed from flow; positioned relative to the **browser viewport**, does not move on scroll.	Work
`sticky`	Behaves as relative until a scroll threshold (e.g. `top: 0`) is reached, then sticks like fixed.	Work (needs at least one)

24.1 z-index

`z-index` sets the stacking order of positioned elements (and flex/grid items). Higher number is in front. It works only when `position` is not static. Values can be negative.

■ Practical Example: Badge on a card, fixed button, sticky header, and an overlay

```
<div class="card">
  <span class="badge">NEW</span>
  Product
</div>
<header class="sticky-header">Sticky header</header>
<a href="#top" class="to-top">Top</a>

.card   { position: relative;           /* reference for the badge */
          width: 240px; height: 160px; background: #eef3ff; margin: 30px; }
.badge  { position: absolute; top: -10px; right: -10px;      /* corner of .card */
          background: crimson; color: white; padding: 4px 10px; border-radius: 12px; }

.sticky-header { position: sticky; top: 0; background: #1b2a49; color: #fff;
                 padding: 14px; z-index: 100; }
.to-top { position: fixed; right: 20px; bottom: 20px; width: 50px; height: 50px;
          border-radius: 50%; background: #2563eb; color: #fff; text-align: center;
          line-height: 50px; z-index: 999; }

/* Perfectly centering with absolute */
.modal { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }

/* Full cover overlay */
/* top/right/bottom/left = 0 */
.overlay { position: absolute; inset: 0; background: rgba(0,0,0,.5); }
```

Result: A red NEW badge hangs on the top-right corner of the card; the header sticks to the top while scrolling; a round Top button stays fixed in the bottom-right corner.

Note: If an absolute element is not placed where you expect, check whether its parent has `position: relative`.

25. Transform Property

`transform` changes the shape/position of an element visually (move, rotate, resize, skew) **without affecting the layout** of surrounding elements. It is GPU-accelerated and great for smooth effects.

25.1 2D transform functions

Function	Effect	Example
`translate(x, y)`, `translateX()`, `translateY()`	Move	`translate(50px, 20px)`
`rotate(angle)`	Rotate clockwise (negative = anticlockwise)	`rotate(45deg)`
`scale(x, y)`, `scaleX()`, `scaleY()`	Resize (1 = normal, 2 = double, 0.5 = half)	`scale(1.2)`
`skew(x-angle, y-angle)`	Slant	`skew(20deg, 10deg)`
`matrix(a,b,c,d,e,f)`	All 2D transforms in one function	`matrix(1,0,0,1,30,40)`

25.2 Related properties

```
transform-origin: top left;      /* pivot point (default: center) */
transform: rotate(30deg) scale(1.2) translateX(20px);   /* multiple: applied right to left */
transform-style: preserve-3d;    /* keep children in 3D space */
perspective: 800px;              /* depth for 3D (set on the parent) */
backface-visibility: hidden;     /* hide the back of a flipped element */
```

25.3 3D functions

`rotateX()`, `rotateY()`, `rotateZ()`, `translateZ()`, `scale3d()`, `rotate3d()`, `perspective()`.

■ Practical Example: Hover effects and a 3D flip card

```
<div class="scene">
  <div class="flip">
    <div class="face front">Front</div>
    <div class="face back">Back</div>
  </div>
</div>

.zoom { transition: transform .3s; }
.zoom:hover  { transform: scale(1.15) rotate(-3deg); }
.moveUp:hover { transform: translateY(-10px); }
.diamond { width: 100px; height: 100px; background: teal; transform: rotate(45deg); }
.slanted { transform: skewX(-15deg); background: #2563eb; color: #fff; padding: 10px 30px; }

/* 3D flip card */
.scene { width: 200px; height: 260px; perspective: 800px; }
.flip  { position: relative; width: 100%; height: 100%; transition: transform .8s;
         transform-style: preserve-3d; }
.scene:hover .flip { transform: rotateY(180deg); }
.face  { position: absolute; inset: 0; display: flex; align-items: center;
         justify-content: center; font-size: 24px; color: #fff; border-radius: 12px;
         backface-visibility: hidden; }
.front { background: #2563eb; }
.back  { background: #16a34a; transform: rotateY(180deg); }
```

Result: Images zoom slightly and tilt on hover; the card flips like a real card to show its back side when the mouse enters.

26. Animation Property

CSS animation lets an element gradually change from one style to another without JavaScript. Two steps: (1) define the animation with `@keyframes`, (2) attach it to an element with `animation` properties.

26.1 @keyframes

```
@keyframes fadeIn {
    from { opacity: 0; }        /* same as 0%   */
    to   { opacity: 1; }        /* same as 100% */
}

@keyframes bounce {
    0%   { transform: translateY(0);     }
    50%  { transform: translateY(-40px); }
    100% { transform: translateY(0);     }
}
```

26.2 Animation properties

Property	Purpose	Values
`animation-name`	Name of @keyframes	`bounce`
`animation-duration`	Time for one cycle (required)	`2s`, `500ms`
`animation-timing-function`	Speed curve	`ease`, `linear`, `ease-in`, `ease-out`, `ease-in-out`, `steps(5)`, `cubic-bezier()`
`animation-delay`	Wait before start	`1s` (negative starts mid-way)
`animation-iteration-count`	Repeats	`1`, `3`, `infinite`
`animation-direction`	Play direction	`normal`, `reverse`, `alternate`, `alternate-reverse`
`animation-fill-mode`	Style before/after	`none`, `forwards`, `backwards`, `both`
`animation-play-state`	Pause / play	`running`, `paused`
`animation`	Shorthand	`name duration timing delay count direction fill-mode`

```
.box { animation: bounce 1.5s ease-in-out 0s infinite alternate both; }
```

26.3 Transition (simple state-change animation)

A transition animates between two states (e.g. normal to hover) and needs a trigger. Properties: `transition-property`, `transition-duration`, `transition-timing-function`, `transition-delay`.

```
.btn { background: #2563eb; transition: background 0.3s ease, transform 0.3s ease; }
.btn:hover { background: #1e40af; transform: scale(1.05); }
```

Note: For smooth, fast animations animate `transform` and `opacity` only. Avoid animating `width`, `height`, `top`, `left`. Respect users who prefer less motion with `@media (prefers-reduced-motion: reduce)`.

■ Practical Example: Loader spinner, pulsing button, sliding text and colour-changing box

```
<div class="spinner"></div>
<button class="pulse">Buy Now</button>
<h2 class="slide">Welcome!</h2>
<div class="magic"></div>

/* 1. Spinner */
.spinner { width: 50px; height: 50px; border: 6px solid #ddd; border-top-color: #2563eb;
           border-radius: 50%; animation: spin 1s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }

/* 2. Pulse */
.pulse { animation: pulse 1.5s infinite; }
@keyframes pulse {
    0%   { box-shadow: 0 0 0 0 rgba(37,99,235,.7); }
    70%  { box-shadow: 0 0 0 20px rgba(37,99,235,0); }
    100% { box-shadow: 0 0 0 0 rgba(37,99,235,0); }
}

/* 3. Slide-in text */
.slide { animation: slideIn 1s ease-out both; }
@keyframes slideIn {
    from { transform: translateX(-100px); opacity: 0; }
    to   { transform: translateX(0);      opacity: 1; }
}

/* 4. Multi-step colour + shape change */
.magic { width: 80px; height: 80px; animation: change 4s infinite alternate; }
@keyframes change {
    0%   { background: red;    border-radius: 0;   }
    50%  { background: gold;   border-radius: 50%; transform: translateX(150px); }
    100% { background: purple; border-radius: 0;   transform: translateX(300px) rotate(180deg); }
}
.stop:hover { animation-play-state: paused; }

@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }
```

Result: A rotating loading ring, a button that sends out pulse waves, a heading that slides in from the left, and a square that moves, rounds and changes colour in a loop.

27. CSS Units

CSS units define lengths (size, spacing, font size). They are of two types: **absolute** and **relative**.

27.1 Absolute units (fixed size)

Unit	Name	Value
`px`	Pixel	Screen dot (1px = 1/96 inch in CSS). Most common
`cm`	Centimetre	1cm = 37.8px
`mm`	Millimetre	1mm = 3.78px
`in`	Inch	1in = 96px = 2.54cm
`pt`	Point	1pt = 1/72 in = 1.33px (used in print)
`pc`	Pica	1pc = 12pt = 16px

27.2 Relative units (depend on something else)

Unit	Relative to	Notes
`%`	Parent's value (width, height, font-size)	Fluid layouts: `width: 50%`
`em`	Font-size of the **current/parent** element	Compounds when nested (2em inside 2em = 4x)
`rem`	Font-size of the **root** `<html>` element	Predictable; default 1rem = 16px; best for fonts and spacing
`vw`	1% of viewport width	`100vw` = full width
`vh`	1% of viewport height	`100vh` = full screen height
`vmin` / `vmax`	1% of smaller / larger viewport side	Responsive squares
`dvh`, `svh`, `lvh`	Dynamic / small / large viewport height	Fix mobile browser bar issue
`ch`	Width of the digit `0` in the font	Good for text width: `max-width: 65ch`
`ex`	Height of letter x	Rarely used
`fr`	Fraction of free space (Grid only)	`grid-template-columns: 1fr 2fr`

27.3 CSS math functions

```
width: calc(100% - 250px);                /* mix units, e.g. page minus sidebar */
width: min(90%, 1000px);                  /* the smaller value */
width: max(300px, 50%);                   /* the larger value  */
font-size: clamp(1rem, 2.5vw, 2rem);      /* min , preferred , max  (fluid text) */
padding: clamp(10px, 3vw, 40px);
```

■ Practical Example: Unit comparison and fluid typography

```
html { font-size: 16px; }                 /* 1rem = 16px */

.a { font-size: 20px; }                   /* absolute                              */
.b { font-size: 1.25rem; }                /* 20px, scales with root setting        */
.c { font-size: 1.25em; }                 /* 1.25 x parent font size               */
.d { width: 50%; }                        /* half of parent                        */
.e { width: 50vw; height: 50vh; }         /* half of screen                        */
.f { max-width: 60ch; }                   /* readable line length                  */
.hero { min-height: 100vh; }              /* fills the entire screen               */
h1 { font-size: clamp(1.8rem, 5vw, 3.5rem); }   /* grows and shrinks with the screen */
.layout { display: grid; grid-template-columns: 250px 1fr; }  /* fixed sidebar + flexible */
```

Tip: Use `rem` for font sizes and spacing, `%`/`fr`/`vw` for layout, `px` for borders and thin lines, `em` for spacing that should scale with the component's own font size.

28. Responsive Design using Media Queries

Responsive web design (RWD) means a website automatically adapts its layout to the size and capabilities of the device (mobile, tablet, laptop, desktop, TV, print). The main ingredients are: (1) the viewport meta tag, (2) flexible layouts (%, fr, flex, grid), (3) flexible images, and (4) **media queries**.

28.1 Viewport meta tag (mandatory)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this tag, mobile browsers pretend to be ~980px wide and shrink the page.

28.2 Media query syntax

```
@media media-type and (media-feature) {
    /* CSS rules that apply only when the condition is true */
}

@media screen and (max-width: 768px) { body { background: lightblue; } }
```

Part	Values
Media type	`all` (default), `screen`, `print`, `speech`
Logical operators	`and`, `,` (comma = OR), `not`, `only`
`width` features	`min-width`, `max-width`, `width`, `min-height`, `max-height`, range syntax `(400px <= width <= 800px)`
`orientation`	`portrait`, `landscape`
`resolution`	`min-resolution: 2dppx` (Retina/HD screens)
`prefers-color-scheme`	`dark`, `light` (dark mode)
`prefers-reduced-motion`	`reduce`
`hover`, `pointer`	`hover: none`, `pointer: coarse` (touch devices)

28.3 Common breakpoints

Device	Width	Query
Small phones	up to 480px	`@media (max-width: 480px)`
Phones / small tablets	481 - 767px	`@media (max-width: 767px)`
Tablets	768 - 1023px	`@media (min-width: 768px)`
Laptops	1024 - 1279px	`@media (min-width: 1024px)`
Desktops / large screens	1280px and above	`@media (min-width: 1280px)`

Note: These are typical only. Best practice is to add a breakpoint **where your design starts to break**, not for specific devices.

28.4 Mobile-first vs Desktop-first

- **Mobile-first (recommended):** write base CSS for phones, then add `min-width` queries for bigger screens. Lighter CSS on mobile, cleaner code.
- **Desktop-first:** write base CSS for desktops, then override for smaller screens using `max-width` queries.

28.5 Responsive images and video

```
img, video { max-width: 100%; height: auto; }          /* never overflow the container */
.cover { width: 100%; height: 250px; object-fit: cover; }
<img srcset="small.jpg 480w, large.jpg 1200w" sizes="(max-width: 600px) 480px, 1200px"
     src="large.jpg" alt="Responsive image">
```

■ Practical Example: Complete responsive page (mobile-first)

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="header">
    <h1 class="logo">MySite</h1>
    <nav class="nav"><a href="#">Home</a><a href="#">About</a><a href="#">Contact</a></nav>
  </header>
  <main class="cards">
    <article class="card">Card 1</article>
    <article class="card">Card 2</article>
    <article class="card">Card 3</article>
    <article class="card">Card 4</article>
  </main>
  <footer class="footer">&copy; 2026 MySite</footer>
</body>
</html>
```

style.css

```
*, *::before, *::after { box-sizing: border-box; }
body   { margin: 0; font-family: Arial, sans-serif; }

/* ---------- Base (mobile first: < 600px) ---------- */
.header { background: #1b2a49; color: #fff; padding: 16px; text-align: center; }
.logo   { margin: 0 0 10px; font-size: 1.5rem; }
.nav a  { display: block; color: #fff; text-decoration: none; padding: 10px; }
.cards  { display: grid; grid-template-columns: 1fr; gap: 16px; padding: 16px; }   /* 1 column */
.card   { background: #dbeafe; padding: 40px 20px; border-radius: 10px; text-align: center; }
.footer { background: #1b2a49; color: #fff; text-align: center; padding: 14px; }

/* ---------- Tablet (>= 600px) ---------- */
@media (min-width: 600px) {
    /* 2 columns */
    .cards { grid-template-columns: repeat(2, 1fr); }
    .nav   { display: flex; justify-content: center; gap: 10px; }
}

/* ---------- Desktop (>= 992px) ---------- */
@media (min-width: 992px) {
    .header { display: flex; justify-content: space-between; align-items: center;
              padding: 16px 60px; }
    .logo   { margin: 0; }
    /* 4 columns */
    .cards  { grid-template-columns: repeat(4, 1fr); max-width: 1200px; margin: 0 auto; }
}

/* ---------- Extras ---------- */
@media (orientation: landscape) and (max-height: 500px) { .header { padding: 6px; } }
@media (prefers-color-scheme: dark) { body { background: #111; color: #eee; } }
@media (hover: none) { .card:hover { transform: none; } }  /* no hover effects on touch */
@media print { .nav, .footer { display: none; } body { color: #000; background: #fff; } }
```

Result: On a phone the four cards are stacked in one column with a vertical menu; on a tablet they form two columns with a horizontal menu; on a desktop they sit in four columns with the logo at left and menu at right. Test it by dragging the browser window narrower, or use DevTools > Toggle Device Toolbar (Ctrl+Shift+M).

28.6 Responsive design checklist

- Add the viewport meta tag.
- Use `box-sizing: border-box` everywhere.
- Use relative units (%, rem, fr, vw) and `max-width` instead of fixed `width`.
- Make images fluid: `max-width: 100%; height: auto;`.
- Use Flexbox (`flex-wrap`) or Grid (`repeat(auto-fit, minmax(220px, 1fr))`) for layouts.
- Design mobile-first; add breakpoints where the layout breaks.
- Make touch targets at least 44 x 44px and text at least 16px.
- Test on real devices and in DevTools device mode.

```
/* Responsive grid with NO media query at all */
.gallery { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 16px; }
```

Quick Revision Cheat Sheet

Topic	Remember
Types of CSS	Inline > Internal/External (later wins) > Browser default; use External
Selectors	`* tag .class #id A B A > B A + B A ~ B [attr] :hover ::before`
Specificity	Inline > ID > Class/Attribute/Pseudo-class > Element
Colours	name, HEX, RGB/RGBA, HSL/HSLA
Box model	content + padding + border + margin; use `box-sizing: border-box`
Margin/Padding shorthand	top right bottom left (clockwise)
Centering	`margin: 0 auto` (block) | `display:flex; justify-content:center; align-items:center`
Position	static, relative, absolute, fixed, sticky (+ z-index)
Display	block, inline, inline-block, none, flex, grid
Transform	translate, rotate, scale, skew (+ transform-origin, perspective)
Animation	`@keyframes` + `animation: name duration timing delay count direction fill`
Units	px (fixed), % , em, rem, vw, vh, fr, calc(), clamp()
Responsive	viewport meta + fluid layout + `@media (min-width: ...)`

Practice assignments

- Build a personal profile card using class/ID selectors, border-radius, box-shadow and hover effects.
- Create a navigation bar with dropdown using combinator and pseudo-class selectors.
- Design a pricing table with three plans; highlight the middle plan using transform: scale().
- Create a hero section with gradient overlay, a fixed background and a call-to-action button.
- Build a loading spinner and a pulsing button using @keyframes.
- Convert any static page into a responsive one that shows 1, 2 and 4 columns at different widths.
- Style a registration form using attribute selectors and :focus, :valid, :invalid states.

Prepared by: Kapil (Mentor)

Keep practising - the best way to learn CSS is to write CSS every day!